ファイルアクセスAPI仕様書

作成日: 2026-02-19 17:05
最終更新: 2026-02-25 15:49
バージョン: 1.1.0
対象画面: 全画面共通
総API数: 8個
実装状況: 8個実装済み、0個未実装

📊 実装状況サマリー

カテゴリ	実装済み	未実装	合計
ファイルアップロード	1	0	1
ファイルダウンロード	4	0	4
バッチ管理	2	0	2
レガシー	1	0	1
合計	**8**	**0**	**8**

完了率: 100% (8/8)

🎯 API一覧

#	API名	メソッド	エンドポイント	状態	優先度
1	ファイルアップロード	POST	`/api/v1/files/upload`	✅ 実装済み	-
2	フォルダZIPダウンロード	POST	`/api/v1/files/download-folder-as-zip`	✅ 実装済み	-
3	署名付きURL取得（ダウンロード用）	POST	`/api/v1/files/download-url`	✅ 実装済み	-
4	署名付きURL取得（表示用）	POST	`/api/v1/files/view-url`	✅ 実装済み	-

5	単一ファイルダウンロード（レガシー）	GET	`/api/v1/projects/:projectId/files/:fileName/download`	✅ 実装済み	-
6	バッチ履歴取得	GET	`/api/v1/files/batches`	✅ 実装済み	-
7	バッチ進捗取得	GET	`/api/v1/files/batches/:batchId/progress`	✅ 実装済み	-

📋 概要

ファイルアクセスAPIは、S3/Swiftストレージへのファイルアップロード・ダウンロード機能を提供します。

主な特徴

1. **汎用性**: プロジェクト、フォーマット、その他あらゆるファイルに対応
2. **バッチ処理**: 大量ファイルのアップロードを効率的に管理
3. **署名付きURL**: セキュアなファイルアクセス
4. **フォルダー一括ダウンロード**: ZIP形式での一括取得

使用箇所

全画面で使用:

- プロジェクト管理画面
- フォーマット管理画面
- 実験記録画面
- データ管理画面

🔐 共通仕様

認証・認可

認証方式: JWT + Session Hybrid

必須ヘッダー:

```
Authorization: Bearer maiml_session-{sessionId}_{timestamp}
```

organization_id の取得:

- ログイン時に `X-Organization-Id` ヘッダーから組織IDを取得し、セッションに保存します
- ログイン後のAPIでは、`Authorization` ヘッダーのセッショントークンから自動的に取得されます
- `authMiddleware` がセッション情報を検証し、ユーザー情報（organization_idを含む）を `req.user` に格納します

権限要件:

- すべてのAPI: すべてのロールがアクセス可能（マルチテナント分離で保護）

ミドルウェアチェーン:

```
router.METHOD('/path',
  authMiddleware,           // JWT認証（セッション検証、req.userに格納）
  upload.array('files'),    // Multerファイルアップロード（アップロード時のみ）
  controller.handler        // コントローラー処理
);
```

マルチテナント分離

- すべてのS3パスに `org_{organization_prefix}` プレフィックス必須
- クロステナントアクセスは厳禁

統一レスポンス形式

成功レスポンス:

```
{
  "success": true,
  "data": { ... },
  "meta": {
    "timestamp": "2026-02-19T17:05:00Z",
    "requestId": "req_abc123"
  }
}
```

エラーレスポンス:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "エラーメッセージ",
    "details": []
  },
  "meta": {
    "timestamp": "2026-02-19T17:05:00Z",
    "requestId": "req_abc123"
  }
}
```

📋 API詳細仕様

1. ファイルアップロード ✅

エンドポイント: `POST /api/v1/files/upload`

説明: S3/Swiftストレージへファイルをアップロード

認証: 必須

権限: すべてのロール

リクエスト

Content-Type: `multipart/form-data`

フォームフィールド:

フィールド	型	必須	説明
`files`	File[]	○	アップロードするファイル（最大100ファイル）
`uploadMode`	string	○	アップロードモード（`single`, `bulk`, `folder`）
`targetPath`	string	○	アップロード先パス（organization_prefixなし）
`metadata`	JSON string	×	メタデータ（JSONオブジェクト）
`emptyFolders`	string[]	×	空フォルダのパス配列

バリデーション:

フィールド	制約
`uploadMode`	`single`, `bulk`, `folder` のいずれか
`targetPath`	1-1024文字
`metadata`	有効なJSONオブジェクト
`emptyFolders`	配列、各要素は文字列
`files`	最大500MB/ファイル、最大100ファイル

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "batchId": "batch-uuid-20260219-170500",
    "uploadedFiles": [
      {
        "originalName": "sample.maiml",
        "s3Key": "org_maiml_lab_001/projects/Sample_Project/sample.maiml",
        "size": 12345,
        "contentType": "application/xml",
        "uploadedAt": "2026-02-19T17:05:00Z"
      }
    ],
    "emptyFolders": [
      "org_maiml_lab_001/projects/Sample_Project/empty_folder"
    ],
    "totalFiles": 1,
```

```
    "totalSize": 12345,
    "status": "completed"
  },
  "meta": {
    "timestamp": "2026-02-19T17:05:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: バリデーションエラー
- 401: 認証エラー
- 413: ファイルサイズ超過
- 500: システムエラー

実装ファイル

- **ルート**: `backend/src/routes/fileAccessRoutes.ts`
- **コントローラー**: `backend/src/controllers/fileAccessController.ts` (`uploadFiles`)
- **サービス**: `backend/src/services/storageService.ts` (`uploadFile`)

実装要件

バッチ管理:

```
// バッチIDを生成
const batchId = `batch-${organizationId}-${Date.now()}`;

// upload_batchテーブルに登録
await db
  .insertInto('upload_batch')
  .values({
    batch_id: batchId,
    organization_id: organizationId,
    user_id: req.user.userId,
    upload_mode: uploadMode,
    target_path: targetPath,
    total_files: files.length,
    uploaded_files: 0,
```

```
    failed_files: 0,
    total_size: totalSize,
    status: 'in_progress',
    metadata: metadata || {},
    create_at: new Date().toISOString()
  })
  .execute();
```

ファイルアップロード処理:

```
for (const file of files) {
  try {
    const s3Key = `${targetPath}/${file.originalname}`;

    await storageService.uploadFile(
      organizationPrefix,
      s3Key,
      file.buffer,
      {
        contentType: file.mimetype,
        metadata: {
          originalName: file.originalname,
          uploadedBy: req.user.userId,
          batchId: batchId
        }
      }
    );

    uploadedFiles.push({
      originalName: file.originalname,
      s3Key: `${organizationPrefix}/${s3Key}`,
      size: file.size,
      contentType: file.mimetype
    });

    // 進捗更新
    await updateBatchProgress(batchId, uploadedFiles.length);
  } catch (error) {
    failedFiles.push({
      fileName: file.originalname,
      error: error.message
    });
  }
}
```

空フォルダプレースホルダー:

```
// Phase1-3: 空フォルダプレースホルダー機能（2026-02-18）
if (emptyFolders && emptyFolders.length > 0) {
  for (const folderPath of emptyFolders) {
    const placeholderKey = `${folderPath}/.placeholder`;

    await storageService.uploadFile(
      organizationPrefix,
      placeholderKey,
      Buffer.from(''),
      {
        contentType: 'text/plain',
        metadata: {
          isPlaceholder: 'true',
          createdBy: req.user.userId
        }
      }
    );
  }
}
```

---

2. フォルダZIPダウンロード ✅

エンドポイント: `POST /api/v1/files/download-folder-as-zip`

説明: 指定フォルダ配下のファイルをZIP形式でダウンロード

認証: 必須

権限: すべてのロール

リクエスト

ボディ（JSON）:

```
{
  "folderPath": "projects/Sample_Project"
}
```

バリデーション:

フィールド	型	必須	制約

`folderPath`	string	○	1-1024文字（organization_prefixなし）

レスポンス

成功時 (200 OK):

- **Content-Type**: `application/zip`
- **Content-Disposition**: `attachment; filename="Sample_Project.zip"`
- **Body**: ZIPバイナリデータ

エラー時:

- 400: バリデーションエラー
- 401: 認証エラー
- 404: フォルダが存在しない
- 500: システムエラー

実装ファイル

- **ルート**: `backend/src/routes/fileAccessRoutes.ts`
- **コントローラー**: `backend/src/controllers/fileAccessController.ts` (`downloadFolderAsZip`)
- **サービス**: `backend/src/services/storageService.ts` (`listFiles`, `downloadFile`)

実装要件

ファイル一覧取得:

```
// フォルダ配下のファイル一覧を取得
const files = await storageService.listFiles(
  organizationPrefix,
  folderPath
);

// .placeholderファイルを除外
const actualFiles = files.filter(file =>
```

```
  !file.key.endsWith('/.placeholder')
);
```

ZIP生成:

```
import archiver from 'archiver';

const archive = archiver('zip', {
  zlib: { level: 9 } // 圧縮レベル
});

// レスポンスヘッダー設定
res.setHeader('Content-Type', 'application/zip');
res.setHeader('Content-Disposition', `attachment; filename="${folderName}.zip"`);

archive.pipe(res);

// ファイルをZIPに追加
for (const file of actualFiles) {
  const fileContent = await storageService.downloadFile(
    organizationPrefix,
    file.key
  );

  // フォルダ構造を維持
  const relativePath = file.key.replace(`${folderPath}/`, '');
  archive.append(fileContent, { name: relativePath });
}

await archive.finalize();
```

---

3. 署名付きURL取得 ✅

エンドポイント: `POST /api/v1/files/download-url`

説明: ファイルキーを指定して署名付きURLを取得

認証: 必須

権限: すべてのロール

リクエスト

ボディ（JSON）:

```
{
  "fileKey": "projects/Sample_Project/sample.maiml"
}
```

バリデーション:

フィールド	型	必須	制約
`fileKey`	string	○	1-1024文字（organization_prefixなし）

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "url":
"https://s3.amazonaws.com/bucket/org_maiml_lab_001/projects/Sample_Project/sample.maiml?X-Amz-
Algorithm=...",
    "expiresAt": "2026-02-19T18:05:00Z",
    "expiresIn": 3600
  },
  "meta": {
    "timestamp": "2026-02-19T17:05:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: バリデーションエラー
- 401: 認証エラー
- 404: ファイルが存在しない
- 500: システムエラー

実装ファイル

- **ルート**: `backend/src/routes/fileAccessRoutes.ts`
- **コントローラー**: `backend/src/controllers/fileAccessController.ts`（`getFileDownloadUrl`）
- **サービス**: `backend/src/services/storageService.ts`（`getSignedUrl`）

実装要件

署名付きURL生成:

```
// ファイルの存在確認
const fileExists = await storageService.fileExists(
  organizationPrefix,
  fileKey
);

if (!fileExists) {
  throw new NotFoundError('ファイルが見つかりません');
}

// 署名付きURLを生成（有効期限: 1時間）
const signedUrl = await storageService.getSignedUrl(
  organizationPrefix,
  fileKey,
  3600
);

const expiresAt = new Date(Date.now() + 3600 * 1000).toISOString();
```

---

4. 署名付きURL取得（表示用） ✅

エンドポイント: `POST /api/v1/files/view-url`

説明: ビューアでのファイル表示用に署名付きURLを取得する。`download-url` との違いは `Content-Disposition: attachment` ヘッダーを付与しない点（ブラウザ上での表示を優先）。

認証: 必須

権限: すべてのロール

追加日: 2026-02-25（ビューアボタン修正タスク）

リクエスト

ボディ（JSON）:

```
{
  "fileKey": "projects/Sample_Project/image.png"
}
```

バリデーション:

フィールド	型	必須	制約
`fileKey`	string	○	1-1024文字（organization_prefixなし）

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "url": "https://swift.example.com/org_maiml_lab_001/projects/Sample_Project/image.png?
temp_url_sig=...",
    "expiresAt": "2026-02-25T16:45:00Z",
    "expiresIn": 3600
  },
  "meta": {
    "timestamp": "2026-02-25T15:45:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: バリデーションエラー（fileKeyが空 / 1024文字超）
- 401: 認証エラー
- 404: ファイルが存在しない
- 500: システムエラー

実装ファイル

- **ルート**: `backend/src/routes/fileAccessRoutes.ts`
- **コントローラー**: `backend/src/controllers/fileAccessController.ts` (`getFileViewUrl`)
- **サービス**: `backend/src/services/fileAccessService.ts` (`getSignedViewUrl`)

フロントエンド使用箇所

コンポーネント	用途
`ImageViewer.tsx`	画像ファイル表示用URL取得
`TiffViewer.tsx`	TIFFファイルダウンロード用URL取得
`ViewRendererDialog.tsx`	MAIMLテキスト取得（URL経由でfetch）

フロントエンドサービス呼び出し例:

```
// frontend/src/services/fileAccessService.ts

// 署名付きURLをそのまま返す（ImageViewer / TiffViewer用）
export async function getFileViewUrl(fileKey: string): Promise<string> {
  const response = await apiClient.post('/files/view-url', { fileKey });
  return response.data.data.url;
}

// テキストとして取得（MAIMLファイル用）
export async function getFileContentAsText(fileKey: string): Promise<string> {
  const url = await getFileViewUrl(fileKey);
  const res = await fetch(url);
  if (!res.ok) throw new Error(`ファイル取得失敗: ${res.status}`);
  return res.text();
}
```

---

5. 単一ファイルダウンロード（レガシー） ✅

エンドポイント: `GET /api/v1/projects/:projectId/files/:fileName/download`

説明: プロジェクトファイルの署名付きURL取得（レガシーエンドポイント）

認証: 必須

権限: すべてのロール

非推奨: `/api/v1/files/download-url` の使用を推奨

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectId`	UUID	○	プロジェクトID
`fileName`	string	○	ファイル名（1-255文字）

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "url": "https://s3.amazonaws.com/...",
    "expiresAt": "2026-02-19T18:05:00Z"
  },
  "meta": {
    "timestamp": "2026-02-19T17:05:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: バリデーションエラー
- 401: 認証エラー
- 404: プロジェクトまたはファイルが存在しない
- 500: システムエラー

実装ファイル

- **ルート**: `backend/src/routes/fileAccessRoutes.ts`
- **コントローラー**: `backend/src/controllers/fileAccessController.ts`（`downloadFile`）

---

6. バッチ履歴取得 ✅

エンドポイント: `GET /api/v1/files/batches`

説明: アップロードバッチの履歴を取得

認証: 必須

権限: すべてのロール

リクエストパラメータ

クエリパラメータ:

パラメータ	型	必須	デフォルト	説明
`status`	string	×	-	ステータスフィルタ（`in_progress`, `completed`, `completed_with_errors`, `failed`）
`startDate`	ISO8601	×	-	開始日時フィルタ
`endDate`	ISO8601	×	-	終了日時フィルタ
`limit`	integer	×	20	取得件数（1-100）
`offset`	integer	×	0	オフセット（0以上）

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
```

```
    "batches": [
      {
        "batch_id": "batch-uuid-20260219-170500",
        "upload_mode": "bulk",
        "target_path": "projects/Sample_Project",
        "total_files": 10,
        "uploaded_files": 10,
        "failed_files": 0,
        "total_size": 123450,
        "status": "completed",
        "create_at": "2026-02-19T17:05:00Z",
        "complete_at": "2026-02-19T17:06:00Z",
        "duration_seconds": 60
      }
    ],
    "totalCount": 50,
    "limit": 20,
    "offset": 0
  },
  "meta": {
    "timestamp": "2026-02-19T17:05:00Z",
    "requestId": "req_abc123"
  }
}
```


エラー時:

- 400: バリデーションエラー
- 401: 認証エラー
- 500: システムエラー

実装ファイル

- **ルート**: `backend/src/routes/fileAccessRoutes.ts`
- **コントローラー**: `backend/src/controllers/fileAccessController.ts`（`getBatchHistory`）
- **サービス**: `backend/src/repositories/uploadBatchRepository.ts`（`list`）

実装要件

DBクエリ:

```
let query = db
  .selectFrom('upload_batch')
  .where('organization_id', '=', organizationId);

// ステータスフィルタ
if (status) {
  query = query.where('status', '=', status);
}

// 日付範囲フィルタ
if (startDate) {
  query = query.where('create_at', '>=', startDate);
}
if (endDate) {
  query = query.where('create_at', '<=', endDate);
}

// ページネーション
const batches = await query
  .selectAll()
  .orderBy('create_at', 'desc')
  .limit(limit)
  .offset(offset)
  .execute();

// 総件数を取得
const totalCount = await query
  .select(db.fn.count('batch_id').as('count'))
  .executeTakeFirstOrThrow();
```

---

7. バッチ進捗取得 ✅

エンドポイント: `GET /api/v1/files/batches/:batchId/progress`

説明: 指定バッチの進捗状況を取得

認証: 必須

権限: すべてのロール

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`batchId`	string	○	バッチID

レスポンス

成功時 (200 OK):


```
{
  "success": true,
  "data": {
    "batch_id": "batch-uuid-20260219-170500",
    "status": "in_progress",
    "total_files": 100,
    "uploaded_files": 45,
    "failed_files": 2,
    "progress_percentage": 45.0,
    "total_size": 1234567,
    "uploaded_size": 555555,
    "estimated_time_remaining": 120,
    "create_at": "2026-02-19T17:05:00Z",
    "last_update_at": "2026-02-19T17:07:00Z"
  },
  "meta": {
    "timestamp": "2026-02-19T17:07:00Z",
    "requestId": "req_abc123"
  }
}
```


エラー時:

- 400: バリデーションエラー
- 401: 認証エラー
- 404: バッチが存在しない
- 500: システムエラー

実装ファイル

- **ルート**: `backend/src/routes/fileAccessRoutes.ts`
- **コントローラー**: `backend/src/controllers/fileAccessController.ts` (`getBatchProgress`)

- サービス: `backend/src/repositories/uploadBatchRepository.ts` (`getById`)

実装要件

進捗計算:

```
const batch = await db
  .selectFrom('upload_batch')
  .where('batch_id', '=', batchId)
  .where('organization_id', '=', organizationId)
  .selectAll()
  .executeTakeFirstOrThrow();

// 進捗率計算
const progressPercentage = (batch.uploaded_files / batch.total_files) * 100;

// 残り時間推定
const elapsedSeconds = (new Date().getTime() - new Date(batch.create_at).getTime()) / 1000;
const filesPerSecond = batch.uploaded_files / elapsedSeconds;
const remainingFiles = batch.total_files - batch.uploaded_files;
const estimatedTimeRemaining = Math.ceil(remainingFiles / filesPerSecond);
```

🔒 セキュリティ考慮事項

マルチテナント分離の徹底

S3パスの必須チェック:

```
// ✅ 良い例
await storageService.uploadFile(
  organizationPrefix,  // organization_prefix必須
  fileKey,
  fileBuffer
);

// ❌ 悪い例
await storageService.uploadFile(
  '',  // organization_prefixなし
```

```
  fileKey,
  fileBuffer
);
```

ファイルサイズ制限

- **単一ファイル**: 500MB
- **バッチ合計**: 制限なし（バッチ管理で制御）
- **理由**: 大容量ファイルの取り扱い
- **超過時**: 413 Payload Too Large

ファイルタイプ検証

MIMEタイプチェック:

```
const allowedMimeTypes = [
  'application/xml',
  'text/xml',
  'application/json',
  'text/plain',
  'image/png',
  'image/jpeg',
  'application/pdf'
];

if (!allowedMimeTypes.includes(file.mimetype)) {
  throw new ValidationError('許可されていないファイルタイプです');
}
```

署名付きURLの有効期限

- **デフォルト有効期限**: 1時間（3600秒）
- **最大有効期限**: 7日間（604800秒）
- **理由**: セキュリティとユーザビリティのバランス

📚 関連ドキュメント

規約

- バックエンド規約
- セキュリティ規約

設計書

- S3構造

📝 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-02-19 17:05	初版作成	Cline
		- fileAccessRoutes.tsの全エンドポイントを反映	
		- 実装済み: 7個（100%完了）	
		- Phase1-2: ファイルサイズ上限500MBに引き上げ	
		- Phase1-3: 空フォルダプレースホルダー機能追加	
		- Phase2-1: バッチ管理機能追加	

API #7: ファイルテキスト内容取得

概要

項目	内容
HTTPメソッド	POST
エンドポイント	`/api/v1/files/content-as-text`

説明	指定したファイルのテキスト内容をS3/Swiftから取得して文字列として返却する
実装状態	✅ 実装済み
権限	すべてのロール（マルチテナント分離で保護）
呼び出し元	fileAccessService.ts (`getFileContentAsText`)

リクエスト

リクエストボディ（JSON）

```
{
  "filePath": "projects/sample_project/sample_project.maiml"
}
```

フィールド	型	必須	説明
filePath	string	✅	organization_id省略の相対ファイルパス

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "data": {
    "content": "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n<maiml>...</maiml>",
    "filePath": "projects/sample_project/sample_project.maiml",
    "contentType": "application/xml",
    "size": 12345
  }
}
```

フィールド	型	説明
content	string	ファイルのテキスト内容
filePath	string	リクエストで指定したファイルパス
contentType	string	ファイルのContent-Type
size	number	ファイルサイズ（バイト）

エラー時

ステータス	エラーコード	説明
400	VALIDATION_ERROR	filePathが未指定
401	AUTH_TOKEN_INVALID	認証エラー
404	FILE_NOT_FOUND	ファイルが存在しない
500	SYSTEM_ERROR	S3/Swiftアクセスエラー

実装要件

- `storageService.downloadFile(organizationId, filePath)` を使用
- organization_idはJWTセッションから自動取得（リクエストに含める必要なし）
- バイナリファイルは文字列変換されるが文字化けする場合あり（テキスト系ファイル向け）

API #8: ファイル削除

概要

項目	内容
HTTPメソッド	DELETE
エンドポイント	`/api/v1/files/delete`
説明	指定したファイルをS3/Swiftから削除する
実装状態	✅ 実装済み
権限	すべてのロール（マルチテナント分離で保護）
呼び出し元	fileAccessService.ts (`deleteFile`)

リクエスト

リクエストボディ（JSON）

```
{
  "filePath": "projects/sample_project/archive/sample_project_v1.maiml"
```

```
}
```

フィールド	型	必須	説明
filePath	string	✅	organization_id省略の相対ファイルパス

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "message": "ファイルを削除しました",
  "data": {
    "filePath": "projects/sample_project/archive/sample_project_v1.maiml"
  }
}
```

エラー時

ステータス	エラーコード	説明
400	VALIDATION_ERROR	filePathが未指定
401	AUTH_TOKEN_INVALID	認証エラー
403	FORBIDDEN	他組織ファイルへのアクセス
404	FILE_NOT_FOUND	ファイルが存在しない
500	SYSTEM_ERROR	S3/Swiftアクセスエラー

実装要件

- `storageService.deleteFile(organizationId, filePath)` を使用
- organization_idはJWTセッションから自動取得
- マルチテナント分離：organization_idがパスに自動付与されるため他組織ファイルへのアクセス不可
- file_metadataテーブルが存在する場合は連動して削除

変更履歴（追記分）

バージョン	日付	変更内容	担当者
1.1.0	2026-03-06	ProjectManagement画面調査によりfileAccessService.tsから呼び出されるAPI追記	Cline
		- API #7: ファイルテキスト内容取得 POST /api/v1/files/content-as-text	
		- API #8: ファイル削除 DELETE /api/v1/files/delete	

作成者: Cline **作成日**: 2026-02-19 17:05 **最終更新**: 2026-03-06 15:37